

Day 3 — 标准库兵器谱（完整版）

一、collections — 比内置容器更强大的工具

1. 是什么

`collections` 是 Python 标准库中提供"增强版容器"的模块。它填补了内置类型（list、dict、set、tuple）在实际工程中的常见缺口。

常用的类：

类名	一句话	替代谁
<code>defaultdict</code>	访问不存在的 key 时不报错，自动创建	dict
<code>Counter</code>	计数+排序一步到位	手动 dict 计数
<code>deque</code>	两端操作都是 O(1) 的列表	list（当队列用）
<code>namedtuple</code>	有字段名的元组	普通元组、简单类
<code>ChainMap</code>	多层字典合并查找	多个字典
<code>OrderedDict</code>	记住插入顺序（3.7 后 dict 也记住了）	普通 dict

2. 解决了什么问题

问题 1：字典分组时要重复检查 **key** 存在

```
# 不用 defaultdict 的写法
word_counts = {}
for word in words:
    if word not in word_counts: # 每次都检查
        word_counts[word] = 0
```

```

    word_counts[word] += 1

# 更糟的是把值也做成列表
groups = {}
for item in items:
    key = item["category"]
    if key not in groups:
        groups[key] = []
    groups[key].append(item)

```

这段代码的"先检查再写"模式在 Python 中太常见了，以至于有必要用一个专门的类来消除样板代码。

问题 2：计数场景太频繁

按种类统计、频次分析、排行榜……这些需求写手动 `dict.get(key, 0) + 1` 太重复。

问题 3：list 当队列用性能差

`list.pop(0)` 和 `list.insert(0, x)` 都是 $O(n)$ 操作——每次都会移动所有元素。

问题 4：轻量数据结构要写类

只是想表示一个 (x, y) 点，不想写一个完整的 class——但又想用 `.x` 而不是 `[0]`。

3. 核心理论

defaultdict

```

from collections import defaultdict

# 核心思想：给 dict 一个"工厂函数"，访问不存在的 key 时自动调用它创建默认值
dd = defaultdict(int)
# 等价于：
# class MyDict(dict):
#     def __missing__(self, key):
#         self[key] = int()
#         return self[key]

```

```
dd["a"] += 1 # "a" 不存在 → int() → 0 → +=1 → 1
print(dd)    # defaultdict(<class 'int'>, {'a': 1})
```

工厂函数必须是可调用的无参函数或 `None` :

```
# 常见工厂
defaultdict(int)      # → 0
defaultdict(float)    # → 0.0
defaultdict(str)      # → ""
defaultdict(list)     # → []
defaultdict(set)      # → set()
defaultdict(bool)     # → False
defaultdict(lambda: "N/A") # 自定义

# None 的特殊行为
dd = defaultdict(None)
# print(dd["x"]) # ✕ TypeError: 'NoneType' object is not callable
```

Counter

Counter 是 dict 的子类，所以可以用所有 dict 方法：

```
from collections import Counter

c = Counter("abracadabra")
print(c) # Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})

# dict 方法全能用
for key in c:
    print(key, c[key])

print(c.keys()) # dict_keys(['a', 'b', 'r', 'c', 'd'])
print(c.values()) # dict_values([5, 2, 2, 1, 1])
```

Counter 的独有方法：

```
# most_common — 频率排序
c.most_common() # [('a',5), ('b',2), ('r',2), ('c',1), ('d',1)]
```

```

c.most_common(2)      # [('a',5), ('b',2)]
c.most_common()[::-1]  # 最少的两个

# elements — 按计数展开
list(c.elements())     # ['a','a','a','a','a','b','b','r','r','c','d']

# subtract — 批量减法
c.subtract("car")
print(c) # Counter({'a': 4, 'b': 2, 'r': 1, 'c': 0, 'd': 1})

# total (3.10+) — 元素总数
c.total() # 11 (5+2+2+1+1)

```

Counter 的数学运算：

```

c1 = Counter(a=3, b=1, c=0)
c2 = Counter(a=1, b=2, d=3)

c1 + c2 # Counter({'a': 4, 'b': 3, 'd': 3})  相加（只保留正数）
c1 - c2 # Counter({'a': 2})                  相减（只保留正数）
c1 & c2 # Counter({'a': 1, 'b': 1})          交集（取最小值）
c1 | c2 # Counter({'a': 3, 'b': 2, 'd': 3})  并集（取最大值）
+c1     # Counter({'a': 3, 'b': 1})          移除零和负数
-c1     # Counter()                          只保留负数转正（这里没有）

```

deque — 双端队列

deque 内部用分段数组（**blocks**）实现，两端操作都是 O(1)：

```

from collections import deque

dq = deque(maxlen=3) # 固定大小，超出的自动丢弃
dq.append(1)
dq.append(2)
dq.append(3)
dq.append(4) # 1 自动从左侧弹出
print(dq)    # deque([2, 3, 4], maxlen=3)

```

```

# 两端操作 O(1)
dq.appendleft(0)      # deque([0, 2, 3])
dq.pop()              # 3, deque([0, 2])
dq.popleft()          # 0, deque([2])
dq.extend([4, 5])      # deque([2, 4, 5])
dq.extendleft([1])     # deque([1, 2, 4, 5]) 注意 extendleft 反转输入

# 旋转
dq.rotate(1)          # deque([5, 1, 2, 4]) 向右移
dq.rotate(-1)         # deque([1, 2, 4, 5]) 向左移

```

namedtuple

```

from collections import namedtuple

# 创建 (类名, 字段列表)
Point = namedtuple("Point", ["x", "y"])

# 用法
p = Point(3, 4)
p.x          # 3 — 属性访问
p[0]         # 3 — 索引访问
x, y = p     # 解包

# 关键特性: 不可变
# p.x = 5    # ✖ AttributeError

# 修改需要创建新对象
p2 = p._replace(x=10) # Point(x=10, y=4)
p3 = Point(x=5, y=p.y) # 手动构造

# 转字典
p._asdict() # {'x': 3, 'y': 4}

# 字段信息
p._fields   # ('x', 'y')

```

ChainMap

ChainMap 不合并字典——它在查找时按顺序搜索各个字典：

```
from collections import ChainMap

defaults = {"theme": "light", "lang": "en", "debug": False}
user = {"theme": "dark", "lang": "zh"} # 用户覆盖
runtime = {"debug": True} # 运行时覆盖

config = ChainMap(runtime, user, defaults)
# 查找顺序: runtime → user → defaults

config["theme"] # "dark" (来自 user)
config["debug"] # True (来自 runtime)
config["lang"] # "zh" (来自 user)
config.get("font_size", "12px") # "12px"

# 写入永远影响第一个映射
config["timeout"] = 30 # 写入 runtime
config["theme"] = "light" # 覆盖 runtime 的 theme
del config["theme"] # 从 runtime 删除, 但 user 还在
config["theme"] # "dark" (来自 user)

# new_child - 增加一层
config2 = config.new_child({"debug": False})
# 现在四层: 新字典 → runtime → user → defaults
```

4. 场景

场景 1：树形结构（**defaultdict** 嵌套）

```
from collections import defaultdict
import json

# 用 defaultdict 自动构建嵌套字典
def tree():
    return defaultdict(tree)
```

```

categories = tree()
categories["fruit"]["red"]["apple"] = 3
categories["fruit"]["red"]["strawberry"] = 5
categories["fruit"]["yellow"]["banana"] = 2
categories["animal"]["mammal"]["dog"] = 1

print(json.dumps(categories, indent=2))
# {
#   "fruit": {
#     "red": {"apple": 3, "strawberry": 5},
#     "yellow": {"banana": 2}
#   },
#   "animal": {
#     "mammal": {"dog": 1}
#   }
# }

```

这种模式叫"递归 defaultdict"——访问任意深度的 key 都会自动创建路径上的所有父节点。

场景 2：滑动窗口（**deque**）

```

from collections import deque

class MovingAverage:
    def __init__(self, size: int):
        self.window = deque(maxlen=size)
        self.sum = 0

    def next(self, val: int) -> float:
        if len(self.window) == self.window.maxlen:
            self.sum -= self.window[0] # 减去即将被弹出的
            self.window.append(val)
            self.sum += val
        return self.sum / len(self.window)

ma = MovingAverage(3)
print(ma.next(1)) # 1.0

```

```
print(ma.next(10)) # 5.5
print(ma.next(3)) # 4.666...
print(ma.next(5)) # 6.0 (窗口:[10, 3, 5])
```

场景 3：配置文件层级（**ChainMap**）

```
from collections import ChainMap

# 模拟 bash 的环境变量查找
env = ChainMap()

# 加载不同层级的配置
with open("envs/prod.env") as f:
    env = env.new_child(parse_env_file(f))

with open("secrets.env") as f:
    env = env.new_child(parse_env_file(f))

# 运行时环境变量优先级最高
env = env.new_child(os.environ)

print(env["DATABASE_URL"]) # 自动按优先级查找
```

5. 替代方案对比

场景	不用 collections	用 collections	结论
分组	<code>dict.setdefault</code> / 反复 <code>if key in dict</code>	<code>defaultdict(list)</code>	<code>defaultdict</code> 更短
计数	<code>dict.get(k, 0) + 1</code> / <code>try-except</code>	<code>Counter</code>	<code>Counter</code> 还有排序
双端操作	<code>list.pop(0)</code>	<code>deque.popleft()</code>	$O(1)$ vs $O(n)$
	写 class + <code>__init__</code>		

场景	不用 collections	用 collections	结论
数据类型		<code>namedtuple</code> / <code>dataclass</code>	<code>namedtuple</code> 不可变
多层配置	手动合并 dict	<code>ChainMap</code>	<code>ChainMap</code> 不拷贝

6. 常见坑

```

# 坑 1: defaultdict 的默认值会隐式创建 key
dd = defaultdict(list)
print("a" in dd)    # False — 没访问就不创建
print(dd["a"])      # [] — 创建了
print("a" in dd)    # True — 隐式创建了

# 坑 2: defaultdict 的工厂函数只调用一次
dd = defaultdict(dict)
dd["a"]["x"] = 1    # ✓ 可以, 因为 dict() 返回新 dict
# dd["a"]["y"]["z"] = 2 # ✗ TypeError — "y"不存在

# 坑 3: Counter 访问不存在的 key 返回 0 (不会报错)
c = Counter()
print(c["nonexistent"]) # 0 — 可能隐藏 bug

# 坑 4: deque maxlen 是无边界的上限, 不是固定长度
dq = deque(maxlen=3)
dq.append(1)
dq.append(2)
print(len(dq))    # 2, 还没满
dq.append(3)
print(len(dq))    # 3, 满了
dq.append(4)      # 1 被自动弹出, len 还是 3

# 坑 5: namedtuple 不可变—忘了这点会踩坑
Point = namedtuple("Point", ["x", "y"])

```

```
p = Point(1, 2)
# p.x = 3 # ✖ AttributeError – 要明白这是 tuple
```

7. 代码验证

```
# 验证 ChainMap 不拷贝
from collections import ChainMap

a = {"x": 1}
b = {"y": 2}
cm = ChainMap(a, b)

cm["x"] = 100      # 修改 ChainMap
print(a["x"])      # 100 – 原字典也被改了！

cm.new_child({"z": 3})
print(len(cm.maps)) # 还是 2 – new_child 返回新 ChainMap, 不修改原来的
```

```
# 验证 deque 两端复杂度
from collections import deque
import time

n = 100000

# list 左端插入
lst = []
t0 = time.perf_counter()
for i in range(n):
    lst.insert(0, i)
print(f"list.insert(0): {time.perf_counter() - t0:.3f}s")

# deque 左端插入
dq = deque()
t0 = time.perf_counter()
for i in range(n):
    dq.appendleft(i)
print(f"deque.appendleft: {time.perf_counter() - t0:.3f}s")
```

```
# 实测差距：list 是  $O(n^2)$ ，deque 是  $O(n)$ 
```

二、itertools — 迭代器瑞士军刀

1. 是什么

`itertools` 是 Python 标准库中最被低估的模块之一。它提供了用于构建迭代器管道的工具函数——每个函数都返回一个迭代器（惰性求值），可以组合成数据处理链。

核心思想：不要一次性加载全部数据，用迭代器逐个处理。

2. 解决了什么问题

- 处理大数据时不想全部加载到内存
- 常见的遍历模式（无限序列、组合、分组、变换、切片）需要重复实现
- 嵌套循环的代码又慢又丑

3. 核心理论

所有 `itertools` 函数都返回迭代器——不立即计算，只在被消费时才逐个产生值。这意味着你可以构建无限长的链而不消耗内存。

分类如下：

类别	函数	作用
无限	<code>count</code>	无限计数
无限	<code>cycle</code>	无限循环
无限	<code>repeat</code>	无限重复
组合	<code>product</code>	笛卡尔积
组合	<code>permutations</code>	排列

类别	函数	作用
组合	<code>combinations</code>	组合
组合	<code>combinations_with_replacement</code>	可重复组合
变换	<code>chain</code>	串接迭代器
变换	<code>compress</code>	布尔过滤
变换	<code>dropwhile/takewhile</code>	条件跳过/取
变换	<code>filterfalse</code>	反向过滤
变换	<code>starmap</code>	展开参数映射
变换	<code>accumulate</code>	累积运算
分组	<code>groupby</code>	分组（必须预排序）
切片	<code>islice</code>	迭代器切片
新式	<code>pairwise</code> (3.10+)	相邻对
新式	<code>batched</code> (3.12+)	分批处理

无限迭代器

```

from itertools import count, cycle, repeat

# count(start=0, step=1) – 无限计数
for i in count(start=10, step=2):
    if i > 20:
        break
    print(i)  # 10, 12, 14, 16, 18, 20

# 常见使用：给数据加序号
for idx, item in zip(count(1), data):
    print(idx, item)

```

```

# cycle(iterable) – 无限循环
colors = cycle(["red", "green", "blue"])
for _ in range(5):
    print(next(colors)) # red, green, blue, red, green

# 实战：轮询多个端点
endpoints = cycle(["server1", "server2", "server3"])
for _ in range(10):
    server = next(endpoints)
    check_health(server)

# repeat(object, times=None) – 无限或有限重复
for x in repeat("ping", 3):
    print(x) # ping, ping, ping

# 常用：map 的恒定参数
list(map(pow, range(5), repeat(2)))
# [0, 1, 4, 9, 16] – 相当于 [x**2 for x in range(5)]

```

组合迭代器

```

from itertools import product, permutations, combinations

# product(*iterables, repeat=1) – 笛卡尔积
list(product("AB", [1, 2]))
# [('A', 1), ('A', 2), ('B', 1), ('B', 2)]

# repeat 参数用于自身乘积
list(product("AB", repeat=2))
# [('A', 'A'), ('A', 'B'), ('B', 'A'), ('B', 'B')]

# 替代嵌套循环
# 不用 product:
for x in range(3):
    for y in range(3):
        process(x, y)

# 用 product:

```

```

for x, y in product(range(3), range(3)):
    process(x, y)
# 嵌套越深, product 优势越大

# permutations(iterable, r=None) – 排列
list(permutations("ABC", 2))
# [('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'), ('C', 'A'), ('C', 'B')]
# 注意顺序有关, ('A', 'B') ≠ ('B', 'A')

# combinations(iterable, r) – 组合 (不放回)
list(combinations("ABC", 2))
# [('A', 'B'), ('A', 'C'), ('B', 'C')]

# combinations_with_replacement – 组合 (可重复)
list(combinations_with_replacement("ABC", 2))
# [('A', 'A'), ('A', 'B'), ('A', 'C'), ('B', 'B'), ('B', 'C'), ('C', 'C')]

```

变换与过滤

```

from itertools import chain, compress, dropwhile, takewhile, filterfalse,
starmap, accumulate

# chain – 串接多个可迭代对象
list(chain([1, 2], [3, 4], "ab"))
# [1, 2, 3, 4, 'a', 'b']

# chain.from_iterable – 展开嵌套
list(chain.from_iterable([[1,2], [3,4], [5]]))
# [1, 2, 3, 4, 5]

# compress – 按 mask 过滤 (比列表推导快)
data = "ABCDEF"
selectors = [1, 0, 1, 0, 1, 0]
list(compress(data, selectors))
# ['A', 'C', 'E']

# dropwhile – 跳过开头满足条件的
list(dropwhile(lambda x: x < 5, [1, 4, 6, 3, 7]))
# [6, 3, 7] – 遇到第一个不满足的之后, 后面的都不跳过

```

```

# takewhile – 取开头满足条件的（遇到第一个不满足的停止）
list(takewhile(lambda x: x < 5, [1, 4, 6, 3, 7]))
# [1, 4]

# filterfalse – 过滤器：取不满足条件的
list(filterfalse(lambda x: x % 2, range(10)))
# [0, 2, 4, 6, 8]

# starmap – 解包参数映射
from operator import pow
list(starmap(pow, [(2, 3), (3, 2), (4, 2)]))
# [8, 9, 16] – 相当于 pow(2,3), pow(3,2), pow(4,2)

# accumulate – 累积（默认是累加）
list(accumulate([1, 2, 3, 4, 5]))
# [1, 3, 6, 10, 15]

from operator import mul
list(accumulate([1, 2, 3, 4, 5], mul))
# [1, 2, 6, 24, 120]

```

分组与切片

```

from itertools import groupby, islice, pairwise, batched

# groupby – 相邻分组（必须先排序，否则不准！）
data = [
    ("fruit", "apple"), ("fruit", "banana"),
    ("animal", "dog"), ("fruit", "cherry"),
]
# ⚠ 没排序的结果：
for key, group in groupby(data, key=lambda x: x[0]):
    print(key, list(group))
# fruit [('fruit', 'apple'), ('fruit', 'banana')]
# animal [('animal', 'dog')]
# fruit [('fruit', 'cherry')] ← 又出现一次 fruit！

# ✔ 正确用法：先排序

```

```

sorted_data = sorted(data, key=lambda x: x[0])
for key, group in groupby(sorted_data, key=lambda x: x[0]):
    print(key, len(list(group))) # fruit: 3, animal: 1

# islice – 迭代器切片
for line in islice(open("large.log"), 5): # 前 5 行
    print(line.strip())

# 也支持 start, stop, step
list(islice(range(10), 2, 8, 2)) # [2, 4, 6]

# pairwise (3.10+) – 相邻对
list(pairwise([1, 2, 3, 4]))
# [(1, 2), (2, 3), (3, 4)]

# 实战：检测相邻差值
diffs = [b - a for a, b in pairwise(data)]

# batched (3.12+) – 分批
list(batched(range(10), 3))
# [(0, 1, 2), (3, 4, 5), (6, 7, 8), (9,)]

```

4. 场景

场景 1：逐块读取大文件（**islice** + **batched**）

```

from itertools import islice, batched

def read_in_chunks(filepath, chunk_size=1000):
    """以行组方式读大文件"""
    with open(filepath) as f:
        for chunk in batched(f, chunk_size):
            yield [line.strip() for line in chunk]

# 读前 100 个块
for chunk in islice(read_in_chunks("huge.log"), 100):
    process_chunk(chunk)

```


场景 2：带索引的分页（**count**）

```
from itertools import count, islice

class Paginator:
    def __init__(self, items, page_size=10):
        self._items = items
        self.page_size = page_size

    def get_page(self, page_num):
        start = (page_num - 1) * self.page_size
        return list(
            islice(self._items, start, start + self.page_size)
        )

    def all_pages(self):
        for i in count(1):
            page = self.get_page(i)
            if not page:
                break
            yield i, page
```

场景 3：排列组合测试用例（**product**）

```
from itertools import product

# 生成所有测试参数组合
params = {
    "db": ["sqlite", "postgres", "mysql"],
    "cache": ["redis", "memcached", "none"],
    "debug": [True, False],
}

test_cases = list(product(*params.values()))
print(len(test_cases)) # 3 × 3 × 2 = 18

for db, cache, debug in test_cases:
    run_test(db=db, cache=cache, debug=debug)
```

```
# 用 dict 理解：
for combo in product(*params.values()):
    case = dict(zip(params.keys(), combo))
    run_test(**case)
```

5. 替代方案对比

场景	itertools	替代方案	选哪个
无限序列	count	while 循环	itertools 更干净
嵌套循环	product	多重 for	嵌套 ≤ 2 层时随意， ≥ 3 层用 product
组合/排列	permutations/ combinations	手动递归	永远用 itertools
串接迭代器	chain	list1 + list2	大数据用 chain(惰性)，小数据随意
分组	groupby	defaultdict	需要排序/有序时用 groupby，否则 defaultdict
切片	islice	list slicing	大数据用 islice

6. 常见坑

```
# 坑 1: groupby 必须先排序！
# 见上面的例子，不排序会得到多个相同 key 的组

# 坑 2: itertools 返回迭代器——一次消费就没了
from itertools import product
p = product("AB", [1, 2])
print(list(p)) # [('A', 1), ('A', 2), ('B', 1), ('B', 2)]
print(list(p)) # [] - 耗尽了
```

```

# 坑 3: islice 的负索引和步进是贪婪的
# islice 不支持负索引 (不像列表)
# list(islice(range(10), -3))    # ✕ 无效
# list(islice(range(10), -3, None)) # ✕ 无效

# 坑 4: chain.from_iterable 只展平一层
nested = [[1, 2], [3, [4, 5]]]
list(chain.from_iterable(nested))
# [1, 2, 3, [4, 5]] - [4, 5] 没展开

```

7. 代码验证

```

# 验证: product 的性能优势
from itertools import product
import time

# 4 层嵌套循环
def nested_loops():
    result = []
    for a in range(10):
        for b in range(10):
            for c in range(10):
                for d in range(10):
                    result.append((a, b, c, d))
    return result

def product_loop():
    return list(product(range(10), repeat=4))

t0 = time.perf_counter()
nested_loops()
t1 = time.perf_counter()
product_loop()
t2 = time.perf_counter()

print(f"nested: {t1-t0:.4f}s")
print(f"product: {t2-t1:.4f}s")
# product 通常略快, 但主要优势是代码简洁

```

```
# 验证: islice 的惰性求值
from itertools import islice

def expensive_generator():
    for i in range(10):
        print(f" generating {i}")
        yield i

print("Before islice:")
gen = expensive_generator()
sliced = islice(gen, 2)

print("After islice:")
print(list(sliced)) # 到这里才按需生成

# 输出顺序:
# Before islice:
# After islice:
#   generating 0
#   generating 1
# [0, 1]
```

三、functools — 函数工具箱

1. 是什么

`functools` 是专门对函数进行"元操作"的模块——它接受函数作为输入，返回（通常）另一个函数。

2. 解决了什么问题

- 想固定函数的某些参数（`partial`）
- 想自动缓存函数结果提高性能（`lru_cache` / `cache`）
- 想根据参数类型选择不同实现（`singledispatch`）
- 想累计运算数据（`reduce`，虽然现在有更现代的替代）
- 想保留装饰后函数的元数据（`wraps`）

3. 核心理论

partial — 部分应用（固定参数）

```
from functools import partial

def power(base, exp):
    return base ** exp

# 固定 exp 参数
square = partial(power, exp=2)
cube = partial(power, exp=3)

print(square(5))    # 25
print(cube(5))      # 125
print(square(10))   # 100

# 实战：简化函数调用
def request(method, url, headers=None, timeout=30):
    ...

get = partial(request, "GET")
post = partial(request, "POST")

get("/api/users", timeout=5)    # 不用传 method
post("/api/users", {"name": "Leo"})

# partial 的工作原理（简化版）
def my_partial(func, /, *args, **keywords):
    def wrapper(*fargs, **fkeywords):
        new_keywords = {**keywords, **fkeywords}
        return func(*args, *fargs, **new_keywords)
    return wrapper
```

`partial` 在 Python 中的使用非常频繁——几乎所有带 callback 的库（Tkinter、asyncio、multiprocessing）都有它的身影。

lru_cache / cache — 自动记忆化

```
from functools import lru_cache

@lru_cache(maxsize=128)
def fibonacci(n):
    """斐波那契数列（带缓存）"""
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

# 第一次调用：正常计算
# 后续调用：从缓存读

fibonacci(40) # 瞬间完成
fibonacci.cache_info()
# CacheInfo(hits=38, misses=41, maxsize=128, currsize=41)

@lru_cache(maxsize=None) # 无限制缓存
def expensive_io(path):
    ...

# Python 3.9+ 的简写
from functools import cache
@cache # 等价于 @lru_cache(maxsize=None)
def fib(n):
    ...
```

性能对比：

```
import time

def fib_raw(n):
    return n if n < 2 else fib_raw(n-1) + fib_raw(n-2)

@cache
def fib_cached(n):
    return n if n < 2 else fib_cached(n-1) + fib_cached(n-2)
```

```

t0 = time.perf_counter()
fib_raw(35)
print(f"raw: {time.perf_counter()-t0:.3f}s")

t0 = time.perf_counter()
fib_cached(35)
print(f"cached: {time.perf_counter()-t0:.3f}s")
# raw 可能数秒, cached 瞬间

```

singledispatch — 单分派泛型

```

from functools import singledispatch

@singledispatch
def serialize(obj):
    """默认实现"""
    raise TypeError(f"Unsupported type: {type(obj)}")

@serialize.register
def _(obj: int) -> str:
    return str(obj)

@serialize.register
def _(obj: float) -> str:
    return f"{obj:.2f}"

@serialize.register
def _(obj: str) -> str:
    return f'"{obj}"'

@serialize.register
def _(obj: list) -> str:
    items = ", ".join(serialize(x) for x in obj)
    return f"[{items}]"

@serialize.register
def _(obj: dict) -> str:
    pairs = ", ".join(f"{k}: {serialize(v)}" for k, v in obj.items())
    return f"{{{pairs}}}"

```

```

print(serialize(42))          # "42"
print(serialize(3.14159))     # "3.14"
print(serialize("hello"))     # '"hello"'
print(serialize([1, 2, 3]))   # "[1, 2, 3]"
print(serialize({"a": 1}))    # "{a: 1}"

# 自定义类型一样支持
@serialize.register
def _(obj: datetime) -> str:
    return obj.isoformat()

# 注册 None
@serialize.register(type(None))
def _(obj) -> str:
    return "null"

```

什么时候用 **singledispatch** 而不是 **if-else** : - 类型分支很多 (5+) - 需要第三方扩展注册新类型

wraps — 保留装饰器元数据

```

from functools import wraps

def my_decorator(f):
    @wraps(f) # 没有这一行, add.__name__ 会变成 "wrapper"
    def wrapper(*args, **kwargs):
        print(f"Calling {f.__name__}")
        return f(*args, **kwargs)
    return wrapper

@my_decorator
def add(a, b):
    """Add two numbers"""
    return a + b

print(add.__name__) # "add" (有 @wraps)
print(add.__doc__)  # "Add two numbers"

```


4. 场景

场景 1：API 请求重试 (**partial + decorator**)

```
from functools import partial

def retry(func, max_retries=3):
    for attempt in range(max_retries):
        try:
            return func()
        except Exception:
            if attempt == max_retries - 1:
                raise

def fetch_data(url, params=None):
    ...

# 不修改原函数，创建特定版本
fetch_with_retry = partial(retry, max_retries=5)
safe_fetch = fetch_with_retry(partial(fetch_data, "/api/data"))
```

场景 2：缓存昂贵计算 (**lru_cache**)

```
from functools import lru_cache
import requests

@lru_cache(maxsize=32, ttl=60) # ttl 需要 3.12+, 或自己实现
def get_weather(city: str) -> dict:
    """缓存在 60 秒内相同城市的天气查询"""
    resp = requests.get(f"https://api.weather.com/{city}")
    return resp.json()

# 也可以在运行时查看缓存命中情况
stats = get_weather.cache_info()
print(f"hits={stats.hits}, misses={stats.misses}")
```

```
# 手动清空
get_weather.cache_clear()
```

场景 3：序列化不同类型（**singledispatch**）

```
from functools import singledispatch
from datetime import datetime
from pathlib import Path


@singledispatch
def to_json(obj):
    return str(obj)


@to_json.register
def _(obj: datetime) -> str:
    return obj.isoformat()


@to_json.register
def _(obj: Path) -> str:
    return str(obj)


@to_json.register
def _(obj: Exception) -> str:
    return f"{type(obj).__name__}: {obj}"


# 还能按抽象类型注册
import numbers
@to_json.register
def _(obj: numbers.Real) -> str:
    return f"{obj:.6f}"
```

5. 替代方案对比

场景	functools	替代方案	选哪个
固定参数	partial	lambda	代码复用多时用 partial

场景	functools	替代方案	选哪个
记忆化	cache/ lru_cache	手动 dict 缓存	纯函数用 cache，有大小限制用 lru_cache
类型分派	singledispatch	if/elif/elif/...	类型少用 if，注册模式用 singledispatch
装饰器元数据	wraps	手动复制 <code>__name__</code> / <code>__doc__</code>	永远用 wraps
归约	reduce	for 循环 / sum / 列表推导	3 行以上用 for，否则 reduce

6. 常见坑

```

# 坑 1: partial 固定位置参数要小心
def f(a, b, c):
    pass

p = partial(f, 1, 2)
p(3) # ✓ f(1, 2, 3)

p = partial(f, 1)
p(2, 3) # ✓ f(1, 2, 3)

# 但混用位置和关键字要注意
p = partial(f, 1, c=3)
# p(2, c=4) # ✗ TypeError: got multiple values for 'c'

# 坑 2: lru_cache 的参数必须可哈希
@lru_cache
def process_list(lst):
    # 参数 lst 是 list (不可哈希) —— 会报错
    return sum(lst)
# ✗ TypeError: unhashable type: 'list'
# 改成元组即可: @lru_cache def process_tuple(t: tuple): ...

```

```

# 坑 3: singledispatch 是基于第一个参数的**类型注解**
# 没有类型注解时默认走主函数
@singledispatch
def f(obj):
    return "default"

@f.register
def _(obj):    # ← 没有类型注解！不会注册
    return "never called"

# 坑 4: reduce 代码可读性差
# ✖
from functools import reduce
result = reduce(lambda acc, x: acc | {x: x**2}, data, {})

# ✔
result = {x: x**2 for x in data}

```

7. 代码验证

```

# 验证 partial 与 lambda 的性能差异
from functools import partial
import time

def add(a, b):
    return a + b

# partial
add5_p = partial(add, 5)
# lambda
add5_l = lambda x: add(5, x)

# 功能一样
print(add5_p(3))    # 8
print(add5_l(3))    # 8

# partial 略快 (lambda 要多一层函数调用)
# 且 partial 保留了 func 信息
print(add5_p.func)  # <function add at ...>

```

```
print(add5_p.args)  # (5,)

# lambda 的 __name__ 是 "<lambda>"
```

```
# 验证 lru_cache 与手动缓存的等价性
from functools import lru_cache

call_count = 0

@lru_cache(maxsize=None)
def compute(n):
    global call_count
    call_count += 1
    return n * n

# 第一次：计算
compute(5)  # call_count = 1
compute(5)  # cache hit, call_count = 1
compute(10) # call_count = 2
compute(5)  # cache hit, call_count = 2

print(compute.cache_info())
# CacheInfo(hits=2, misses=2, maxsize=None, currsize=2)
```

四、错误处理模式

1. 是什么

Python 有一套独特的错误处理哲学：**EAFP**（Easier to Ask Forgiveness than Permission，先试再求饶）。

这与其他语言的 LBYL（Look Before You Leap，先看再跳）形成鲜明对比。

2. 解决了什么问题

LBYL 的问题：- 检查和操作之间有竞态条件（检查通过了，操作时变了）- 检查的逻辑和操作的逻辑重复（都在描述同一个条件）- 检查越复杂，代码越长

```
# LBYL 的问题
if os.path.exists("config.json"):
    if os.access("config.json", os.R_OK):
        with open("config.json") as f:
            data = json.load(f)
    else:
        data = {}
else:
    data = {}
# 问题：检查完到打开之间文件可能被删/改权限
```

3. 核心理论

```
# EAFP — Python 推荐风格
try:
    with open("config.json") as f:
        data = json.load(f)
except FileNotFoundError:
    data = {}
except PermissionError:
    data = {}
except json.JSONDecodeError:
    data = {}

# 更简洁
try:
    with open("config.json") as f:
        data = json.load(f)
except (FileNotFoundError, PermissionError, json.JSONDecodeError):
    data = {}
```

异常链（**raise ... from**）：

```

class AppError(Exception):
    """应用层异常"""
    pass

def load_config(path):
    try:
        with open(path) as f:
            return json.load(f)
    except FileNotFoundError as e:
        raise AppError(f"Config not found: {path}") from e
        # "from e" 保留了原始异常堆栈

try:
    load_config("missing.json")
except AppError as e:
    print(e)          # "Config not found: missing.json"
    print(e.__cause__) # FileNotFoundError
    import traceback
    traceback.print_exc()
    # 显示完整异常链

```

不要捕获所有异常：

```

# ✖ 反面教材
try:
    data = process_input(user_input)
except Exception: # 什么异常都吞了
    pass

# ✔ 明确指定
try:
    data = process_input(user_input)
except ValueError:
    data = default
except TypeError:
    data = default

```

4. 场景

场景 1：contextlib.suppress — 优雅忽略异常

```
from contextlib import suppress

# 不用 suppress:
try:
    os.remove("temp.txt")
except FileNotFoundError:
    pass

# 用 suppress:
with suppress(FileNotFoundError):
    os.remove("temp.txt")

# 多个异常
with suppress(FileNotFoundError, PermissionError):
    os.remove("protected.txt")

# 实战：清理临时文件
with suppress(FileNotFoundError):
    os.unlink("/tmp/cache.dat")
with suppress(FileNotFoundError):
    shutil.rmtree("/tmp/build")
```

场景 2：自定义异常体系

```
class APIError(Exception):
    """API 基础异常"""
    def __init__(self, message, status_code=None, response=None):
        super().__init__(message)
        self.status_code = status_code
        self.response = response

class NotFoundError(APIError):
    """资源不存在"""
    pass
```



```

class RateLimitError(APIError):
    """限流"""
    def __init__(self, message, retry_after=60):
        super().__init__(message, status_code=429)
        self.retry_after = retry_after

class AuthError(APIError):
    """认证失败"""
    pass

# 使用
try:
    response = api.get_user(42)
except NotFoundError:
    print("User not found, create one")
except RateLimitError as e:
    print(f"Rate limited, retry after {e.retry_after}s")
except AuthError:
    print("Check API key")

```

5. 常见坑

```

# 坑 1: except 的顺序会匹配
try:
    process()
except Exception:          # 先捕获所有异常
    print("caught")
except ValueError:         # 永远不会到这里
    print("never")

# ✔ 先具体后通用
try:
    process()
except ValueError:
    print("Value error")
except TypeError:
    print("Type error")
except Exception:         # 兜底

```

```

    print("Other error")

# 坑 2: try 块越大越好? 不一定
# 太大: 不知道哪一行出的错
try:
    data = load_file()
    parsed = parse(data)
    result = process(parsed)
    save(result)
except Exception:
    pass # 什么错了都不知道

# 太小: 代码充斥 try/except
try:
    data = load_file()
except Exception:
    ...

try:
    parsed = parse(data)
except Exception:
    ...

# ✔ 黄金法则: 一个 try 块做「一件事情」
try:
    data = load_file()
except FileNotFoundError:
    data = default

try:
    result = process(parse(data))
except ValueError:
    result = default

```

6. 代码验证

```

# 验证 EAFP 优于 LBYL 的场景
import time

# 准备数据: 10 个 key 中 3 个不存在

```

```
data = {i: i * 10 for i in range(7)}
keys = list(range(10))

# LBYL
def lbyl():
    result = []
    for k in keys:
        if k in data:
            result.append(data[k] * 2)
    return result

# EAFP
def eafp():
    result = []
    for k in keys:
        try:
            result.append(data[k] * 2)
        except KeyError:
            pass
    return result

print(lbyl())
print(eafp())
# 对存在的 key, EAFP 比 LBYL 快 (少一次查找)
# 对不存在的 key, EAFP 有异常开销
# 但大多数情况下, "不存在"是异常情况, EAFP 更合理
```

五、logging — 专业日志

1. 是什么

`logging` 是 Python 标准库的日志框架, 它提供了比 `print()` 专业得多的日志能力——包括级别过滤、输出目标、格式控制、日志轮转等。

2. 解决了什么问题

- `print()` 的输出和生产日志混在一起, 不知道哪个是调试信息

- 没有级别控制——debug 和 error 混在一起，无法按严重程度过滤
- 输出目标单一——`print()` 只能到 stdout，不能同时写文件和发邮件
- 没有格式化——纯文本，缺少时间戳、行号、模块名
- 大型应用无法统一日志配置

3. 核心理论

日志级别（由低到高）

级别	数值	什么时候用
DEBUG	10	详细信息，仅开发调试
INFO	20	确认程序正常运行
WARNING	30	发生了不寻常但不是错误的事
ERROR	40	出错了但程序还能继续
CRITICAL	50	严重错误，程序可能得退出

设置 `level=logging.INFO` 后，低于此级别（DEBUG）的消息会被忽略。

基本用法

```
import logging

# 一次性配置（建议在程序入口做）
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
    filename="app.log",      # 写到文件（不写就输出到 stderr）
    filemode="a",           # 追加模式
)

# 获取 logger（用 __name__ 是标准惯例）
logger = logging.getLogger(__name__)
```

```
# 五种级别
logger.debug("Detailed info for debugging")
logger.info("Server started on port %d", 8080) # 支持 % 格式化
logger.warning("Disk space low: %.1f GB", 1.5)
logger.error("Failed to connect to database: %s", err)
logger.critical("System is out of memory!")
```

logger 的层级结构

```
# logger 按名字分层, 用点号分隔
root = logging.getLogger() # 根 logger
app = logging.getLogger("app") # "app"
db = logging.getLogger("app.db") # "app.db" → app 的子 logger
api = logging.getLogger("app.api") # "app.api"

# 子 logger 默认继承父 logger 的设置
# 但可以独立设置 level、handler、filter

for name in ["app", "app.db", "app.api"]:
    logger = logging.getLogger(name)
    print(f"{name}: level={logger.level}, propagate={logger.propagate}")
```

Handler — 控制输出目标

```
import logging

logger = logging.getLogger("myapp")
logger.setLevel(logging.DEBUG)

# Handler 1: 控制台
console = logging.StreamHandler()
console.setLevel(logging.INFO)
console.setFormatter(logging.Formatter(
    "%(asctime)s %(message)s", datefmt="%H:%M:%S"
))

# Handler 2: 文件 (每日轮转)
```

```

from logging.handlers import TimedRotatingFileHandler
file_handler = TimedRotatingFileHandler(
    "app.log",
    when="midnight",      # 每天零点轮转
    backupCount=7,        # 保留 7 天
    encoding="utf-8",
)
file_handler.setLevel(logging.DEBUG)
file_handler.setFormatter(logging.Formatter(
    "%(asctime)s [%(levelname)s] %(name)s: %(lineno)d: %(message)s"
))

# Handler 3: 错误邮件（生产环境）
from logging.handlers import SMTPHandler
mail_handler = SMTPHandler(
    mailhost=("smtp.gmail.com", 587),
    fromaddr="bot@example.com",
    toaddrs=["admin@example.com"],
    subject="App Error Alert",
    credentials=("user", "pass"),
    secure=(),
)
mail_handler.setLevel(logging.ERROR)

# 注册所有 handler
logger.addHandler(console)
logger.addHandler(file_handler)
logger.addHandler(mail_handler)

```

4. 场景

场景 1：结构化 JSON 日志

```

import logging
import json

class JsonFormatter(logging.Formatter):
    def format(self, record):
        log_entry = {

```

```

        "timestamp": self.formatTime(record),
        "level": record.levelname,
        "logger": record.name,
        "message": record.getMessage(),
    }
    # 额外字段
    if hasattr(record, "user_id"):
        log_entry["user_id"] = record.user_id
    if hasattr(record, "request_id"):
        log_entry["request_id"] = record.request_id
    return json.dumps(log_entry, ensure_ascii=False)

# 使用
logger = logging.getLogger("api")
handler = logging.StreamHandler()
handler.setFormatter(JsonFormatter())
logger.addHandler(handler)

logger.info("User logged in", extra={"user_id": 42, "request_id":
"abc-123"})
# {"timestamp": "2026-05-23 11:00:00", "level": "INFO", ...}

```

场景 2：异常日志 + 堆栈追踪

```

import logging

logger = logging.getLogger(__name__)

try:
    1 / 0
except ZeroDivisionError:
    # exc_info=True 自动记录完整堆栈
    logger.exception("Division failed") # 等价于 error + exc_info=True
    # 或：
    logger.error("Division failed", exc_info=True)

```

场景 3：性能日志

```
import logging
import time
from contextlib import contextmanager

logger = logging.getLogger("performance")

@contextmanager
def time_log(name):
    start = time.perf_counter()
    try:
        yield
    finally:
        elapsed = time.perf_counter() - start
        if elapsed > 1.0:
            logger.warning("Slow operation: %s took %.3fs", name, elapsed)
        else:
            logger.debug("Operation: %s took %.3fs", name, elapsed)

# 使用
with time_log("database_query"):
    results = db.query("SELECT ...")
```

5. 替代方案对比

场景	logging	替代方案	结论
简单调试	print()	logger.debug	开发用 print，生产用 logging
结构化日志	logging + JSONFormatter	structlog / loguru	小项目用标准库够
性能敏感	控制日志级别	去掉日志	生产环境 INFO 级别
	远程 handler		配合标准格式输出

场景	logging	替代方案	结论
集中式 日志		ELK / Loki / Datadog	

6. 常见坑

```
# 坑 1: 多次调用 basicConfig 无效
logging.basicConfig(level=logging.INFO) # ✔ 第一次
logging.basicConfig(level=logging.DEBUG) # ✘ 无效

# 解决方案: 提前配置, 或显式配置 logger
logger.setLevel(logging.DEBUG)

# 坑 2: 字符串格式化浪费性能
# ✘ 即使 level=WARNING, 字符串已拼接
logger.debug(f"Processing {len(data)} records")

# ✔ 用 % 格式化——logger 内部按 level 过滤后再拼接
logger.debug("Processing %d records", len(data))

# 坑 3: 异常日志不要手动传 traceback
try:
    1/0
except:
    # ✘ 多余
    logger.error(traceback.format_exc())
    # ✔ logging 自带
    logger.exception("Division error")
```

7. 代码验证

```
# 验证日志级别过滤
import logging

logging.basicConfig(level=logging.WARNING)
logger = logging.getLogger(__name__)
```

```
logger.debug("debug")      # x 不输出
logger.info("info")        # x 不输出
logger.warning("warn")     # ✓ 输出
logger.error("error")      # ✓ 输出
logger.critical("crit")    # ✓ 输出
```

六、正则表达式速查

1. 是什么

`re` 模块是 Python 的正则表达式引擎，用于字符串的模式匹配和替换。它兼容 Perl 风格的语法。

2. 解决了什么问题

- 验证格式（邮箱、手机号、URL）
- 提取特定模式的数据（从日志里提取 IP、日期）
- 批量替换（清理文本脱敏）
- 分割字符串（复杂分隔符）

3. 核心理论

基础匹配

```
import re

# match - 从开头匹配
re.match(r"\d+", "123abc")  # <Match object> - 匹配到 "123"
re.match(r"\d+", "abc123")  # None - 开头不是数字

# search - 搜索任意位置
re.search(r"\d+", "abc123def")  # <Match object> - 匹配到 "123"

# fullmatch - 完全匹配
```

```

re.fullmatch(r"\d+", "123")    # <Match object>
re.fullmatch(r"\d+", "123a")   # None

# findall - 找所有匹配
re.findall(r"\d+", "a1b22c333") # ['1', '22', '333']

# finditer - 迭代匹配结果
for m in re.finditer(r"\d+", "a1b22c333"):
    print(m.group(), m.start(), m.end())
# 1 1 2
# 22 3 5
# 333 6 9

```

编译复用

```

# 多次使用相同模式时，编译后性能更好
pattern = re.compile(r"\b\w+@\w+\.\w+\b")

# 编译后的 pattern 有所有方法
pattern.search(text)
pattern.findall(text)
pattern.sub("[REDACTED]", text)

# 性能对比
import time

text = "test@example.com " * 1000

# 每次重新编译
t0 = time.perf_counter()
for _ in range(100):
    re.findall(r"\b\w+@\w+\.\w+\b", text)
print(f"uncompiled: {time.perf_counter() - t0:.3f}s")

# 编译后复用
t0 = time.perf_counter()
pattern = re.compile(r"\b\w+@\w+\.\w+\b")
for _ in range(100):

```

```
pattern.findall(text)
print(f"compiled: {time.perf_counter() - t0:.3f}s")
```

分组

```
# 普通分组
m = re.search(r"(\d{4})-(\d{2})-(\d{2})", "Date: 2026-05-23")
m.group(0)    # "2026-05-23" — 完整匹配
m.group(1)    # "2026"
m.group(2)    # "05"
m.group(3)    # "23"
m.groups()    # ('2026', '05', '23')

# 命名分组 — 可读性更好
m = re.search(
    r"(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})",
    "Date: 2026-05-23"
)
m.group("year")    # "2026"
m.groupdict()      # {'year': '2026', 'month': '05', 'day': '23'}

# 不捕获分组
re.findall(r"(?:Mr|Ms|Dr)\.?\s+(\w+)", "Mr. Smith and Dr. Jones")
# ['Smith', 'Jones']
```

替换

```
# 简单替换
re.sub(r"\d+", "NUMBER", "abc123def456")
# "abcNUMBERdefNUMBER"

# 函数替换
def mask_credit_card(match):
    digits = match.group()
    return "****-****-****-" + digits[-4:]

re.sub(r"\d{4}-\d{4}-\d{4}-\d{4}", mask_credit_card,
    "Card: 1234-5678-9012-3456")
```

```
# "Card: ****-****-****-3456"

# 引用分组
re.sub(r"(\w+)@(\w+\.\w+)", r"\1 at \2",
       "Contact: user@example.com")
# "Contact: user at example.com"
```

常用模式速查

.	任意字符（除了换行）
\d	数字 [0-9]
\w	字母/数字/下划线 [a-zA-Z0-9_]
\s	空白符（空格、Tab、换行）
\b	单词边界
^	字符串开头
\$	字符串结尾
*	0 或多次
+	1 或多次
?	0 或 1 次
{n}	恰好 n 次
{n,}	至少 n 次
{n,m}	n 到 m 次
[abc]	a 或 b 或 c
[^abc]	不是 a/b/c
	或
()	分组
(?:)	不捕获分组
(?P<name>)	命名分组

4. 场景

场景 1：解析日志

```
import re

LOG_PATTERN = re.compile(
    r"(?P<ip>\d+\.\d+\.\d+\.\d+)\s+"
```

```

    r"(?P<ident>\S+)\s+(?P<auth>\S+)\s+"
    r"\[(?P<datetime>[^\]]+)\]\s+"
    r'"(?P<method>\w+)\s+(?P<path>\S+)\s+(?P<proto>\S+)"\s+'
    r"(?P<status>\d{3})\s+(?P<size>\d+)"
)

def parse_access_log(line):
    m = LOG_PATTERN.match(line)
    if m:
        return m.groupdict()
    return None

# 使用
log_line = '192.168.1.1 - - [23/May/2026:10:30:45 +0000] "GET /api/users
HTTP/1.1" 200 1234'
print(parse_access_log(log_line))
# {'ip': '192.168.1.1', 'datetime': '23/May/2026:10:30:45 +0000',
#   'method': 'GET', 'path': '/api/users', 'status': '200', ...}

```

场景 2：数据脱敏

```

import re

def mask_sensitive(text):
    """脱敏手机号和邮箱"""
    # 手机号中间四位
    text = re.sub(r"(1[3-9]\d)\d{4}(\d{4})", r"\1****\2", text)
    # 邮箱用户名部分
    text = re.sub(r"(\w)(\w+)(@)", lambda m: m.group(1) + "****" +
m.group(3), text)
    return text

print(mask_sensitive("Contact: 13812345678, email@test.com"))
# "Contact: 138****5678, e****@test.com"

```

场景 3：模板替换

```
import re

def render_template(template, context):
    """简单的模板引擎：{{ name }} → value"""
    def replacer(match):
        key = match.group(1).strip()
        return str(context.get(key, match.group(0)))
    return re.sub(r"\{\{(\w+)\}\}", replacer, template)

template = "Hello {{ name }}, your order #{ order_id } is ready."
context = {"name": "Leo", "order_id": "12345"}
print(render_template(template, context))
# "Hello Leo, your order #12345 is ready."
```

5. 替代方案对比

场景	正则	替代方案	结论
固定字符串查找	re	<code>str.find()</code> , "abc" in text	用字符串方法
简单替换	re.sub	<code>str.replace()</code>	用 replace
复杂解析	命名分组	手写解析器	日志/配置用正则
HTML 解析	re	BeautifulSoup / lxml	用专用解析器
JSON 提取	re	json 模块	用 json

6. 常见坑

```
# 坑 1: 贪婪匹配
re.findall(r"<.*>", "<div><span>text</span></div>")
# ['<div><span>text</span></div>'] - 匹配了整个字符串

# ✔ 非贪婪: 加 ?
re.findall(r"<.*?>", "<div><span>text</span></div>")
```

```

# ['<div>', '<span>', '</span>', '</div>']

# 坑 2: 转义
# 匹配小数点
re.search(r"\d+\.\d+", "3.14") # ✓ \. 转义
re.search(r"\d+.\d+", "3a14") # ✓ 但 . 没转义, 匹配了任意字符

# 在字符串中反斜杠也要转义
# 所以要用 r 原始字符串

# 坑 3: 回溯灾难
# 这个模式在处理长字符串时非常慢
pattern = re.compile(r"(a+)+b")
# pattern.match("a" * 30) # 可能卡住
# 解决方案: 避免嵌套量词

# 坑 4: ^ 和 $ 的多行模式
re.search(r"^abc", "123\nabc") # None

# ✓ 用 re.MULTILINE
re.search(r"^abc", "123\nabc", re.MULTILINE) # 匹配

# 坑 5: 反斜杠在 str 和 re 中的双重转义
# 匹配 \n 换行
re.search(r"\\n", "hello\\nworld") # ✓ raw string
re.search("\\\\n", "hello\\nworld") # ✓ 但难读

```

7. 代码验证

```

# 验证编译 vs 不编译的性能
import re
import time

# 准备数据
text = "user@example.com " * 10000
pattern_str = r"\b[\w.+~]+@[~\w-]+\.[\w.]+\b"
compiled = re.compile(pattern_str)

```



```
# 每次编译
t0 = time.perf_counter()
for _ in range(100):
    re.findall(pattern_str, text)
print(f"uncompiled: {time.perf_counter() - t0:.3f}s")

# 预编译
t0 = time.perf_counter()
for _ in range(100):
    compiled.findall(text)
print(f"compiled: {time.perf_counter() - t0:.3f}s")

# 差异在高频使用时会很大
```

总结

模块	核心价值	最常用的功能	一句话记法
collections	增强版容器	defaultdict、Counter、deque	不再手动写 "if key in dict"
itertools	迭代器管道	product、chain、groupby、islice	用它替代嵌套循环
functools	函数元操作	partial、cache、wraps	固定参数、缓存结果
contextlib	简化异常处理	suppress	用 with 代替 try-except
logging	专业日志	getLogger、basicConfig	别用 print 了
re	模式匹配	compile、findall、sub	先编译再匹配

明天预告：Day 4 — OOP 深入 & 类型系统（*metaclass* 入门、描述器、ABC/Protocol、*dataclass*、类型注解）